

מדריך מאוחד - סוגי בדיקות ובדיקות ביצועים

29 בספטמבר 2026 · איחוד החומר מארבע שיחות: ראיון רפאל, ארכיטקטורת בדיקות בפיתוח, רמות וסוגי בדיקות, ובדיקות API.

רמות הבדיקה

רמות הבדיקה הן שכבות, מהיחידה הבודדת ועד המערכת המלאה. הן עונות על השאלה "איפה מתחילים לבדוק", ולא על השאלה "מה בודקים".

רמה	מה נבדק	בידוד	דוגמה
בדיקת יחידה - Unit	פונקציה או מחלקה אחת	mocks לכל תלות חיצונית	validateID(id) מחזירה False לפורמט שגוי
בדיקת אינטגרציה - Integration	הממשק בין שני רכיבים אמיתיים	בלי mocks בממשק הנבדק	קורא DB דרך TCP, קורא RS-485 שער דרך
בדיקת מערכת - System / E2E	הזרימה המלאה במערכת השלמה	סביבה מלאה	תג DB → שער → חיישן → רישום
בדיקת קבלה - Acceptance / UAT	האם הדרישה העסקית מתקיימת	סביבה קרובה לייצור	תלוש שכר נכון בסוף חודש

באינטגרציה מתגלים הבאגים שקשה למצוא ביחידה: פרוטוקול, תזמון ותנאי מרוץ. ביחידה כל התלויות מזויפות, ולכן באג בממשק עצמו לא יכול להתגלות שם.



פירמידת הבדיקות · ככל שהרמה נמוכה יותר, הבדיקות מהירות ויציבות יותר

הפירמידה היא המלצה על כמות: הרבה בדיקות ברמות הנמוכות, מעט מאוד ברמה העליונה. פירמידה הפוכה, שבה רוב הבדיקות הן E2E, מייצרת סוויטה איטית ושבירה. מיקרוסופט מסמנת את אותו רעיון כ-L0 עד L4, וגוגל כ-Small, Medium ו-Large.

סוגי הבדיקות

סוגי הבדיקות הם זוויות, לא שכבות. מבחן ההבחנה: דרישה שמתחילה ב"המערכת צריכה ל..." היא בדרך כלל פונקציונלית, ודרישה שמתחילה ב"המערכת צריכה להיות..." (מהירה, זמינה, מאובטחת) היא לא-פונקציונלית. פונקציונלי נמדד בכך או לא, לא-פונקציונלי נמדד במספר מול סף.

בדיקות פונקציונליות

סוג	מה בודק	דוגמה בחניון חכם
Functional	כל פיצ'ר עובד לפי הדרישה	רכב מורשה, המחסום נפתח
Boundary	קצוות של טווחים	נשאר מקום אחד, נשארו 0 מקומות
Negative	קלט לא חוקי ותרחישים חריגים	לוחית שלא קיימת במאגר
Equivalence Partitioning	נציג אחד מכל קבוצת קלט שמתנהגת אותו דבר	מורשה / לא מורשה / חסום
State Transition	מעברים בין מצבים	"סגור" באמצע פתיחה
Integration	החיבור בין רכיבים	מצלמה ← מאגר ← מחסום
End-to-End	תרחיש משתמש מלא	כניסה, חניה, יציאה, חיוב
Regression	שינוי חדש לא שבר משהו קיים	אחרי עדכון גרסה
Smoke / Sanity	בדיקה מהירה לפני בדיקות מעמיקות	המערכת עולה ומזהה לוחית אחת

בדיקות לא-פונקציונליות

סוג	מה בודק	דוגמה בחניין חכם
Performance	זמן תגובה ותפוקה	זיהוי ופתיחה תוך 2 שניות
Load	התנהגות בעומס הצפוי	שעת שיא בבוקר
Stress	התנהגות מעבר לעומס, ואיפה נקודת השבירה	פי 3 מהעומס הרגיל
Spike	עלייה פתאומית בעומס	סיום משחק באצטדיון סמוך
Soak / Endurance	יציבות לאורך זמן רב	שבוע רצוף בלי אתחול
Scalability	האם הוספת משאבים מגדילה את הקיבולת	הוספת שיערים וחניונים
Reliability / Recovery	התאוששות מכשלים	חזרה אחרי נפילת חשמל
Availability	אחוז הזמן שהמערכת זמינה	99.9%
Security	הרשאות, הצפנה, התקפות	זיוף לוחית, גישה לא מורשית למאגר
Safety	מניעת פגיעה פיזית	המחסום לא יורד על רכב
Usability	קלות שימוש	הודעה ברורה לנהג למה לא נפתח
Compatibility	עבודה על סביבות שונות	סוגי מצלמות, פורמטי לוחיות

שתי הבחנות ששואלים עליהן בריאיון

Smoke מול Sanity. Smoke היא רחבה ורדודה, אחרי build חדש: "המערכת בכלל עולה ולא קורסת?". Sanity היא צרה ועמוקה, אחרי תיקון נקודתי: "השינוי הספציפי הזה עובד?".

Negative-ו Boundary הן טכניקות, לא קטגוריות. בדרך כלל מיישמים אותן על בדיקות פונקציונליות, אבל אפשר גם על לא-פונקציונליות: Boundary על ביצועים הוא עומס בדיוק בגבול ה-SLA.

בדיקות ביצועים: משפחה, לא בדיקה אחת

בדיקות ביצועים הן מונח מטרייה. בדיקת עומס היא סוג אחד בתוכה, לצד מאמץ, קפיצה, סיבולת, מדרגיות, נפח וקיבולת. כל סוג עונה על שאלה אחרת לגבי עומס וזמן.

סוג	השאלה שהוא עונה עליה	דוגמה במכשיר קשר
בדיקת עומס - Load	האם המערכת עומדת בדרישות בעומס הצפוי?	30 מכשירים בקבוצה, העיכוב הקולי מתחת לסף
בדיקת מאמץ - Stress	איפה נקודת השבירה, ואיך המערכת נשברת?	60 מכשירים בקבוצה שמוגבלת ל-30
בדיקת קפיצה - Spike	האם המערכת שורדת עלייה פתאומית ומתאוששת?	כל המכשירים לוחצים PTT באותה שנייה
בדיקת סיבולת - Soak	האם היא יציבה לאורך שעות או ימים?	72 שעות רצוף, בלי דליפת זיכרון
בדיקת מדרגיות - Scalability	האם הוספת משאבים משפרת ביצועים בהתאם?	הכפלת שרתים מול הכפלת משתמשים
בדיקת נפח - Volume	איך המערכת מתנהגת עם כמות נתונים גדולה?	חישוב שעות לחודש שלם של 5,000 עובדים
בדיקת קיבולת - Capacity	מה המקסימום שהמערכת מחזיקה בתוך ה-SLA?	כמה מכשירים לפני שהעיכוב חוצה את הסף
בדיקת בו-זמניות - Concurrency	מה קורה כשהרבה פעולות רצות במקביל?	תנאי מרוץ, נעילות, תורים

ההבדל בין מאמץ לקיבולת עדין וחשוב: קיבולת מחפשת את המספר הגבוה ביותר שעדיין עומד בדרישה, מאמץ מחפש את ההתנהגות מעבר לו. השאלה במאמץ היא לא רק מתי המערכת נשברת אלא איך: ירידה מבוקרת בביצועים, או קריסה.

דליפת זיכרון - Memory Leak

תוכנה שתופסת זיכרון ולא משחררת אותו. בבדיקה של דקה לא רואים את זה, אחרי שלושה ימים המכשיר קורס. זו הסיבה שבדיקת סיבולת היא לא מותרות.

מה מודדים

חמישה מדדים מכסים כמעט כל בדיקת ביצועים. מודדים אותם יחד, כי כל אחד לבד מטעה.

מדד	מה הוא	למה הוא לבד לא מספיק
זמן תגובה - Latency	מרגע שליחת הבקשה ועד קבלת התשובה	זמן תגובה מצוין לא שווה כלום אם 5% מהבקשות נכשלות
תפוקה - Throughput	כמה פעולות ביחידת זמן: בקשות, הודעות או פריימים לשנייה	תפוקה גבוהה יכולה להתקבל דווקא מתורים ארוכים
שיעור שגיאות - Error Rate	אחוז הבקשות שנכשלו	בלי latency לא יודעים אם ההצלחות היו בזמן
חוסר יציבות בעיכוב - Jitter	כמה זמן התגובה משתנה בין מדידה למדידה	עיכוב ממוצע תקין יכול להסתיר קפיצות שהורסות קול ווידאו
ניצול משאבים - Resource Utilization	מעבד, זיכרון, רשת, דיסק, ובנייד גם סוללה	אומר למה הביצועים ירדו, לא רק שהם ירדו

ב-jitter כדאי לזכור את הדוגמה: עיכוב קבוע של 200ms נשמע טבעי, ועיכוב שקופץ בין 50 ל-400ms נשמע קטוע. במערכות זמן אמת חוסר היציבות מזיק יותר מהעיכוב עצמו.

את ניצול המשאבים מודדים **במקביל** לבדיקה ולא אחריה. אחרת יש מספר ביצועים בלי הסבר.

איך מודדים נכון

זה החלק שמבדיל בין מי שהריץ כלי לבין מי שיודע לבדוק ביצועים. חמישה עקרונות.

1. מתחילים מדרישה מספרית

"המערכת צריכה להיות מהירה" היא לא דרישה שאפשר לבדוק. "זמן תגובה עד 300ms ב-95% מהבקשות, בעומס של 30 מכשירים" היא כן. אם אין דרישה כזו, זו שאלת ההבהרה הראשונה שלך.

יעד רמת שירות - SLO

המספר שהמערכת מתחייבת אליו: סף, אחוזון, ותנאי העומס שבו הוא נמדד. בלעדיו אין קריטריון מעבר או כישלון.

2. אחוזונים, לא ממוצע

זו הטעות הנפוצה ביותר, וכדאי מאוד להזכיר אותה בריאיון.

אחוזון - Percentile

האחוזון ה-95 הוא הערך שמתחתיו נמצאות 95% מהמידות. כותבים p95.

ממוצע מסתיר את הזנב. אם 99 בקשות חזרו ב-10ms ואחת ב-3 שניות, הממוצע נראה מצוין והמשתמש ה-100 חווה מערכת תקועה. מדווחים p50, p95 ו-p99, וכשיש דרישת זמן אמת גם את המקסימום.

3. מודל פתוח, לא סגור

מודל סגור - Closed Model

כל "משתמש וירטואלי" שולח בקשה, ממתינ לתשובה ורק אז שולח את הבאה. כשהמערכת מאטה, קצב הבקשות יורד מעצמו והעומס בפועל קטן.

מודל פתוח - Open Model

הבקשות נשלחות בקצב קבוע שהוגדר מראש, בלי קשר לקצב שבו המערכת עונה. זה מה שקורה במציאות.

הבעיה במודל סגור נקראת השמטה מתואמת - Coordinated Omission: המחולל "משתף פעולה" עם ההאטה ולכן לא מודד אותה. ב-k6 הפתרון הוא executors מסוג arrival-rate.

4. משוויים לבסיס, לא רק לסף

בדיקה שעברה את הסף אבל האטה ב-40% לעומת הגרסה הקודמת מסמנת רגרסיה, גם אם היא ירוקה. שומרים תוצאת בסיס - Baseline לכל גרסה, ומשוויים אליה.

5. מתעדים את הסיביבה ובודקים את המחולל

תוצאת ביצועים בלי תיאור הסיביבה היא מספר חסר משמעות. מתעדים: גרסת קוד, חומרה, טופולוגיית רשת, נתוני עומס, ומשך הריצה. לפני שמאשימים את המערכת בודקים שהמחולל עצמו לא צוואר הבקבוק: מריצים אותו מול יעד ריק או מול loopback ורואים שהוא מגיע לקצב המבוקש. מריצים כל תרחיש כמה פעמים, כי מדידה אחת היא רעש.

היסטוגרמה בדיוק גבוה - HdrHistogram

מבנה נתונים שמשמר את הזנב במקום למחוק אותו בעיגול, ולכן מאפשר לדווח p99 ו-p99.9 אמינים.

הכלי נבחר לפי השכבה שנבדקת. כלי עומס לאפליקציות ווב לא בודק חומרה, ומחולל תעבורה לא בודק לוגיקה עסקית.

כלי	שכבה	מתי משתמשים בו
JMeter	פרוטוקול: HTTP, TCP, JDBC, JMS	עומס על שירותים ו-API; הוא לא דפדפן ולא מריץ JavaScript של הדף
k6	HTTP ו-gRPC, תסריטים ב-JavaScript	עומס ב-CI, עם executors מסוג arrival-rate למודל פתוח
Locust	HTTP, תסריטים בפייטון	כשרוצים להישאר בפייטון ולשלב עם קוד הבדיקות הקיים
iperf3	רשת גולמית: TCP ו-UDP	מידת רוחב פס, איבוד חבילות ו-jitter בין שתי נקודות
TRex	מחולל תעבורה בקצב גבוה	עומס ברמת החבילות על ציוד רשת ומערכות משובצות
Scapy	בניית חבילות בפייטון	תעבורה מותאמת ופרוטוקולים לא סטנדרטיים
tc netem	הדמיית תנאי רשת	הזרקת עיכוב, jitter ואיבוד חבילות מכוונים
HdrHistogram	ניתוח תוצאות	אחוזי זנב אמינים במקום ממוצעים

במערכות משובצות, רדיו ותקשורת רוב העבודה היא לא JMeter אלא מחוללי תעבורה, סביבת חומרה בלולאה - Hardware in the Loop, ורתמות בדיקה ייעודיות בפייטון או ב-C. המדדים שם הם jitter, latency, איבוד חבילות ודטרמיניזם של תזמון.

שווה להכיר גם שני פרויקטי קוד פתוח ללמידה: labgrid לשליטה בחומרה מפייטון, ו-pytest-embedded של Expressif לבדיקות על התקנים אמיתיים.

מה אומרים התקנים והחברות

ציטוט מקור אחד בריאיון מבדיל בין דעה לבין ידע. אלה המקורות שכדאי להכיר.

מקור	מה הוא אומר
ISTQB, סילבוס CT-PT	בדיקות ביצועים הן מונח מטרייה לכל בדיקה שממוקדת בביצועים תחת נפחי עומס שונים. בדיקת עומס היא אחד משמונה סוגים שם
מילון המונחים של ISTQB	בדיקת עומס מוגדרת מפורשות כסוג של בדיקת ביצועים
ISO/IEC 25010:2023	אין בו "בדיקות ביצועים" בכלל. יש מאפיין איכות בשם performance efficiency עם שלושה תת-מאפיינים: התנהגות בזמן, ניצול משאבים וקיבולת
מיקרוסופט, Azure Well-Architected	בדיקות ביצועים הן פרקטיקה לא-פונקציונלית להערכת התנהגות עומס עבודה בתנאים שונים. הטבלה שלהם: load, stress, spike, soak
k6 / Grafana	במודל סגור, כשהמערכת מאטה קצב הבקשות יורד מעצמו. זו ה-coordinated omission, והפתרון הוא executors מסוג arrival-rate
JMeter, אתר רשמי	הכלי עובד ברמת הפרוטוקול ולא מריץ JavaScript של דפי HTML

ההבחנה הכי חשובה מהטבלה: בדיקות הן הדרך לאמת את מאפיין האיכות. ISO מגדיר מה נדרש, ISTQB מגדיר איך בודקים את זה.

שתי גישות ארגוניות

מיקרוסופט - Shift Left רומות **L0 עד L4**. L1 הן בדיקות מהירות בזיכרון בלי תלויות חיצוניות, L2 ומעלה דורשות סביבה פרוסה. הכלל: כמה שיותר בדיקות ברמות הנמוכות, כי הן מהירות ויציבות.

תפקיד SDET לעומת Combined Engineering: בעבר היה תפקיד ייעודי שכותב תשתיות בדיקה, והיום מיקרוסופט מחזיקה מודל שבו אותו מפתח כותב את הפיצ'ר ואת הבדיקות שלו.

גוגל - Test Sizes. במקום Unit/Integration/E2E, גוגל מסווגת לפי משאבים: Small רץ בתהליך אחד בלי רשת ובלי דיסק, Medium מותר לו localhost, Large מותר לו מערכת מלאה. הסיווג קובע timeout ומה מותר לבדיקה לגעת בו. התפקיד המקביל ל-SDET שם נקרא SET.

בדיקות ברמת API

שכבת ה-API היא המקום שבו כדאי למקם את רוב האוטומציה: הבדיקות שם מהירות ויציבות יותר מבדיקות GUI, ועדיין בודקות התנהגות אמיתית של המערכת. בדיקות GUI ומקצה לקצה נשמרות לתרחישים המרכזיים.

סוג בדיקה	מה בודקים
פונקציונלי	קוד סטטוס, גוף התשובה, כותרות
CRUD	כל פעולה, כולל אימות תופעות הלוואי שלה במאגר
שלילי	400, 401, 403, 404, 422, 500 בתרחישים המתאימים
הרשאות	אימות וזיהוי: מי מורשה לגשת למה
חוזה וסכמה	ולידציה מול JSON Schema, כדי לתפוס שינויי חוזה
ערכי קצה	גבולות של טווחים ואורכי שדות
שרשרת בקשות	תרחיש מלא: תשובה של בקשה אחת מזינה את הבאה
מונחה נתונים	הרצת אותה בדיקה על CSV או JSON של מקרים
סף ביצועים	בדיקה בסיסית שזמן התגובה מתחת לסף

ב-Postman כותבים את הבדיקות ב- `pm.test`, ומריצים את ה-Collection ב-Cl דרך Newman. בפיתון אותו דבר נעשה עם `requests` ו-`pytest`, וזו הגרסה שמשתלבת טוב יותר עם שאר קוד האוטומציה: אותם `fixtures`, אותו דיווח, אותו `pipeline`.

רשימת בדיקה ומשפטי מפתח

לפני הגשת בדיקת ביצועים לסקירה

- יש SLO מספרי: סף, אחוזון ותנאי עומס
- המידה מדווחת p95, p50 ו-p99, לא ממוצע
- המחולל עובד במודל פתוח, או שמתועד למה לא
- נבדק שהמחולל עצמו לא צוואר הבקבוק
- יש השוואה לתוצאת בסיס, לא רק לסף
- ניצול משאבים נמדד במקביל לבדיקה
- הסביבה מתועדת בתוצאה: גרסה, חומרה, טופולוגיה, משך
- כל תרחיש רץ יותר מפעם אחת

משפטים שאפשר להגיד בריאיון

על אסטרטגיית אוטומציה: "את רוב האוטומציה אני כותבת ברמת ה-API והפרוטוקול, כי שם הבדיקות מהירות ויציבות. בדיקות GUI ומקצה לקצה אני שומרת לתרחישים המרכזיים בלבד, ובדיקת עשן רצה ראשונה על כל גרסה חדשה."

על ההבדל בין ביצועים לעומס: "בדיקות ביצועים הן מטרייה, ובדיקת עומס היא סוג אחד בתוכה. ככה זה מוגדר ב-ISTQB וגם ב-Azure Well Architected של מיקרוסופט."

על מדידה: "אני לא מדווחת ממוצע. ממוצע מסתיר את הזנב, ודווקא הזנב הוא מה שהמשתמש מרגיש. אני מדווחת p95 ו-p99 מול סף שהוגדר מראש."

על coordinated omission: "אם המחולל מחכה לתשובה לפני שהוא שולח את הבקשה הבאה, הוא מוריד את העומס בדיוק כשהמערכת מאטה, ולכן לא מודד את ההאטה. לכן משתמשים במודל פתוח."